EC4307 | WEB APPLICATION DEVELOPMENT

STUDYMATE HUB

Course Note-Sharing Web Application

STUDENT	Chamuditha K.D.T.
INDEX NUMBER	EG/2023/5537
SUBMISSION DATE	27 October 2026
GITHUB REPOSITORY	StudyMate Note-Sharing Platform

Project repository
github.com/kaluarachchiteshan-prog/StudyMate-Note-Sharing-platform

A responsive browser-based application for discovering and managing course notes, with complete persistent CRUD backed by MongoDB.

1. Executive summary

StudyMate Hub is a browser-based course-note sharing application. It helps students find resources by course, subject, university, title, or tag, and lets them create, view, edit, and delete notes.

The runnable implementation uses a MERN-style architecture: MongoDB stores note records, Express and Node.js expose a REST API, and React with Vite renders a responsive desktop and mobile web interface. Mongoose defines the data model, and Zod validates API input.

The main project objective is to demonstrate complete, persistent CRUD on a useful academic entity. Unlike a UI-only prototype, note changes are written to and read back from MongoDB. The directory also offers search, subject filtering, and sorting.

2. Problem, objectives, and scope

Students often store course resources across disconnected folders and messages. That makes relevant notes hard to find and makes sharing updates difficult. StudyMate Hub provides a single searchable directory for shared course-note metadata and optional public resource links.

Objectives

- Implement persistent create, read, update, and delete operations for study notes.
- Make notes discoverable through search, subject filters, course codes, and sorting.
- Validate user input in the browser and again at the server.
- Keep the interface responsive and provide visible loading, empty, error, and confirmation states.
- Document the data model, setup, API, and demonstration workflow.

Project scope

The assessed, database-backed entity is Study Note. Notes include academic details, descriptive text, tags, optional file metadata, contributor information, and engagement counters. Authentication, authorization, binary file upload, and group management are outside the current implementation scope. The local classroom demonstration should use non-sensitive sample data.

3. Requirements and architecture

Functional requirements

1. List and view notes stored in MongoDB. 2. Create a note with required academic metadata and optional tags or resource link. 3. Edit an existing note and persist the changes. 4. Confirm and permanently delete a selected note. 5. Search notes and filter by subject; sort by recent, popularity, downloads, or title. 6. Reject invalid data with a useful response and show request failures in the interface.

Architecture

```
Student's web browser
        |
        | React UI, Vite development server (localhost:5173)
        | /api requests proxied during development
        v
Express / Node.js REST API (localhost:5000)
        |
        | Mongoose schema and queries
        v
MongoDB: studymate.notes
```

In development, Vite serves the browser application and proxies `/api` requests to Express. In production, Express serves the built Vite application and the API from one server. Secrets and database connection settings are read from a local server environment file, not embedded in the frontend.

Technology choices

Layer	Technology	Responsibility
Browser UI	React 18, TypeScript, Vite 7	Forms, note directory, details, and responsive presentation
API	Node.js, Express 4, TypeScript	REST endpoints, filtering, validation responses, and health check
Data access	Mongoose 8	Schema, timestamps, indexes, queries, and MongoDB connection
Database	MongoDB	Persistent note records
Input validation	Zod 4	Strict server-side request schema
Styling	Tailwind CSS 3 and custom CSS	Layout, responsive styling, and component states

4. Database design

The database is named `studymate`; the primary collection is `notes`. Each note is an independent document with MongoDB `_id` as its primary key. Mongoose maintains `createdAt` and `updatedAt` timestamps. There are no foreign-key relationships in this focused CRUD scope.

```
+----------------------------+
| STUDY_NOTE                 |
+----------------------------+
| _id                ObjectId PK
| title              String (required, max 120)
| description        String (required, max 1000)
| subject            String (required, max 80)
| courseCode         String (required, max 30)
| university         String (required, max 120)
| tags               Array[String], max 10 entries
| fileUrl            String, optional HTTP(S) URL
| fileName           String, optional
| fileSize           String, optional
| fileType           Enum: pdf, doc, text
| previewText        String, optional
| authorId           String
| authorName         String
| authorUniversity   String
| upvotes            Number, minimum 0
| downvotes          Number, minimum 0
| downloadsCount     Number, minimum 0
| commentsCount      Number, minimum 0
| createdAt          Date
| updatedAt          Date
+----------------------------+
```

An index supports newest-first listing. A text index is defined for note content fields; directory search uses escaped, case-insensitive substring matching over title, description, subject, course code, and tags. Sort options are allow-listed by the API.

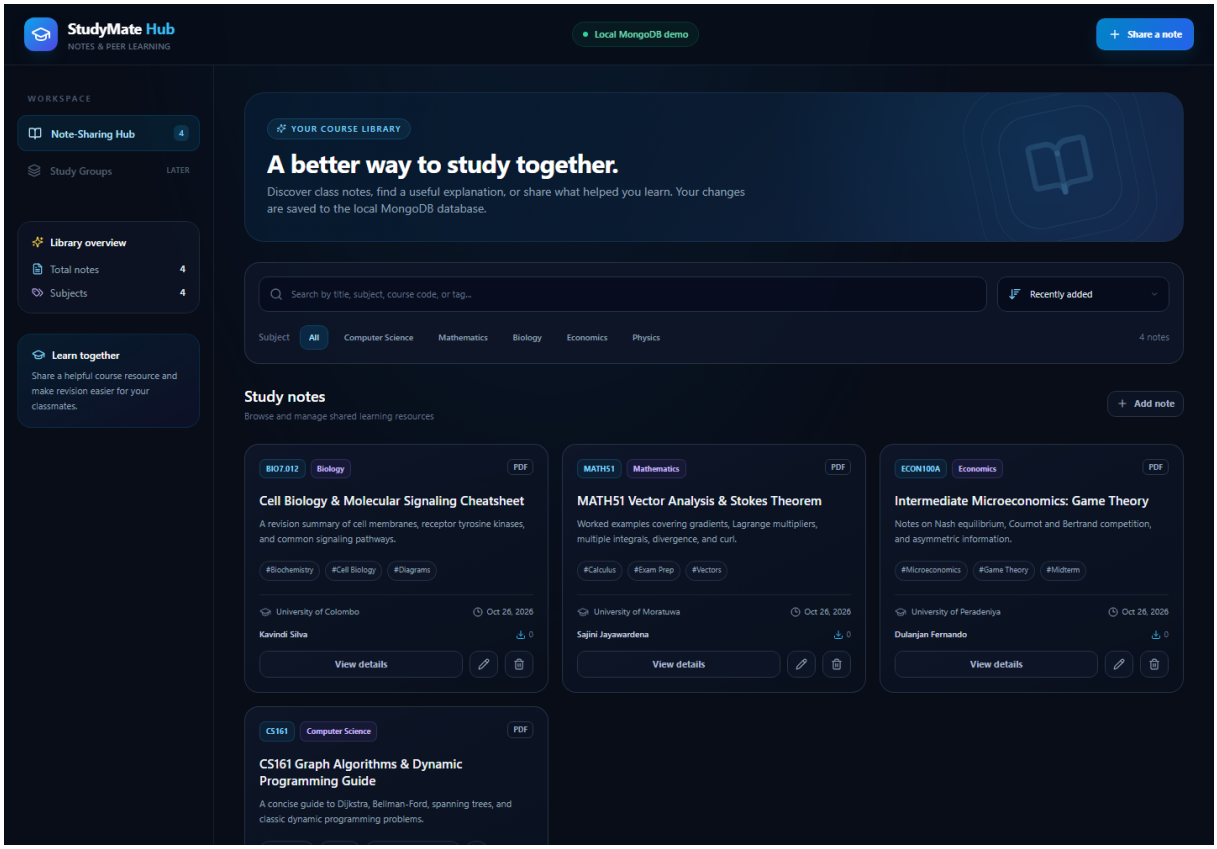
5. CRUD and validation

Operation	REST route	Result
Create	`POST /api/notes`	Validates and inserts a note; returns HTTP 201
Read list	`GET /api/notes`	Lists notes, with optional search, subject, course, and sort query parameters
Read one	`GET /api/notes/:id`	Returns a single note, or 404 when it does not exist
Update	`PUT /api/notes/:id`	Validates and updates an existing note
Delete	`DELETE /api/notes/:id`	Deletes an existing note after user confirmation

The browser form checks required fields, minimum lengths, URL syntax, and maximum lengths. The API independently validates the request with a strict Zod schema, so direct API calls cannot bypass required-field checks. It rejects unexpected properties, invalid URL protocols, and more than ten tags. Invalid input and malformed record IDs return HTTP 400; valid but nonexistent IDs return HTTP 404. The client presents load errors, save errors, retry, loading, empty-state, success, and delete-confirmation feedback.

6. User interface

The primary screen presents a desktop dashboard with navigation, library overview, searchable note cards, subject filters, and a sort selector. Note cards link to a detail dialog and expose explicit edit and delete controls. Create and edit forms share a validated form layout. At smaller widths the sidebar is hidden and the content reflows into a single-column mobile browser layout.



StudyMate Hub desktop notes dashboard

Figure 1. Desktop browser view of the note directory. The responsive layout is also checked at narrow mobile-browser widths; the deliverable remains a website and does not require installing a native mobile application.

7. Verification and results

The application was checked in the local development environment using a MongoDB database, the API, and a browser.

Check	Result
API health and MongoDB connection	Passed; database reported connected
Create / read / update / delete	Passed using both REST calls and the browser UI
Persistence and filtering	Passed; updated note remained readable and searchable before deletion
Invalid note request	Passed; API returned HTTP 400
Missing-note request	Passed; API returned HTTP 404
Server validation tests	Passed: 3 tests for trimming, required fields, unexpected properties, URL schemes, and tag limits
TypeScript checks	Passed for client and server
Production Vite build	Passed
Dependency audit	Passed; zero reported vulnerabilities at verification time
Responsive viewport	Checked at desktop and narrow mobile-browser sizes

Re-run the checks from the repository root with `npm test`, `npm run typecheck`, and `npm run build`.

8. Setup, limitations, and conclusion

Local setup

1. Install Node.js 20.19+ (or 22.12+) and MongoDB Community Server. 2. Run `npm install` in the project root. 3. Copy `.env.example` to `server/.env`. 4. Start MongoDB on the local default address, `mongodb://127.0.0.1:27017`. 5. Run `npm run dev` from the project root. 6. Open `http://localhost:5173`.

The server migrates existing legacy StudyMate notes in place to the current note format. On an empty database, four sample notes are inserted by default. Set `SEED_DEMO_DATA=false` in `server/.env` to disable demo seeding.

Limitations and future work

The application is a teaching/demo project. It does not provide production authentication or authorization, uploaded-file storage, abuse moderation, or deployment hardening. Resource links are optional metadata; this implementation does not upload binary documents. These features would be necessary before using real student accounts or sensitive materials.

Conclusion

StudyMate Hub demonstrates the rubric's central database-backed CRUD requirement with a React web interface, Express REST API, MongoDB persistence, validation, search, filtering, sorting, and documented setup. The included test results and demo workflow make the operations straightforward to verify and present.

Project repository: github.com/kaluarachchiteshan-prog/StudyMate-Note-Sharing-platform